⚙️ Technology Stack

FlightFinder - Complete Technical Architecture

1. MERN Stack Overview

```
┌──────────────────────────────────────────────────────────┐
│                        MERN STACK                        │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  │
│  │   MongoDB    │  │  Express.js  │  │   React.js   │  │
│  │   Database   │  │   Backend    │  │   Frontend   │  │
│  └──────────────┘  └──────────────┘  └──────────────┘  │
│         │                 │                 │          │
│         └─────────────────┴─────────────────┘          │
│                           │                            │
│                      ┌────▼────┐                       │
│                      │ Node.js │                       │
│                      │ Runtime │                       │
│                      └─────────┘                       │
└──────────────────────────────────────────────────────────┘
```

2. Frontend Technologies

React.js v18.2.0

Purpose: UI Library for building interactive user interfaces

Key Features Used:

- Component-based architecture
- React Hooks (useState, useEffect, useContext)
- Virtual DOM for efficient rendering
- JSX syntax for component creation

Why Chosen: Fast, scalable, and has large community support

React Router v6.x

Purpose: Client-side routing and navigation

Key Features Used:

- Route protection and authentication
- Dynamic routing with parameters
- Navigation between pages
- useNavigate and useParams hooks

Context API

Purpose: State management across components

Implementation:

- GeneralContext for global state
- User authentication state
- Booking data management
- Eliminates prop drilling

Axios v1.x

Purpose: HTTP client for API requests

Features:

- Promise-based HTTP requests
- Request and response interceptors

- Error handling
- JSON data transformation

Bootstrap 5 v5.0.2

Purpose: CSS framework for responsive design

Components Used:

- Grid system for layout
- Form controls and inputs
- Buttons and navigation
- Responsive utilities

3. Backend Technologies

Node.js v14+

Purpose: JavaScript runtime environment

Features:

- Event-driven, non-blocking I/O
- NPM package management
- ES6+ module support
- Asynchronous programming

Why Chosen: JavaScript on both frontend and backend, high performance

Express.js v4.18.2

Purpose: Web application framework

Features Used:

- RESTful API routing
- Middleware support
- Request/response handling
- Error handling

bcrypt v5.1.0

Purpose: Password hashing and encryption

Implementation:

- Hash passwords with salt rounds (10)
- Compare hashed passwords for login
- Secure password storage

Security: Industry-standard encryption algorithm

CORS v2.8.5

Purpose: Cross-Origin Resource Sharing

Configuration:

- Allow requests from React frontend
- Configure allowed origins

- Handle preflight requests

Body-Parser v1.20.2

Purpose: Parse incoming request bodies

Features:

- JSON body parsing
- URL-encoded data parsing
- Request size limits

4. Database Technology

MongoDB v4.4+

Purpose: NoSQL document database

Features:

- Document-oriented storage (JSON-like)
- Flexible schema design
- High performance and scalability
- Rich query language

Collections:

- users - User accounts and authentication
- flights - Flight information
- bookings - Ticket bookings

Mongoose v7.4.1

Purpose: MongoDB object modeling (ODM)

Features Used:

- Schema definition and validation
- Model creation and queries
- Middleware and hooks
- Data relationships (ObjectId references)

5. Development Tools

Tool	Purpose	Usage
npm	Package manager	Install and manage dependencies
Git	Version control	Source code management
VS Code	Code editor	Development environment
Postman	API testing	Test backend endpoints
Chrome DevTools	Browser debugging	Frontend debugging and testing

6. System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                        CLIENT TIER                           │
│  ┌──────────────────────────────────────────────────────────┐  │
│  │  React.js Application (Port 3000)                        │  │
│  │  - Components (Login, Register, Navbar, etc.)            │  │
│  │  - Pages (LandingPage, BookFlight, Admin, etc.)          │  │
│  │  - Context API (State Management)                        │  │
│  │  - React Router (Navigation)                             │  │
│  │  - Bootstrap 5 (Styling)                                 │  │
│  └──────────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
                            │
                            │ HTTP/HTTPS (Axios)
                            │
                            ▼
┌──────────────────────────────────────────────────────────────┐
│                       APPLICATION TIER                       │
│  ┌──────────────────────────────────────────────────────────┐  │
│  │  Node.js + Express.js Server (Port 6001)                 │  │
│  │  - RESTful API Endpoints                                 │  │
│  │  - Authentication & Authorization                        │  │
│  │  - Business Logic                                        │  │
│  │  - Request Validation                                    │  │
│  │  - Error Handling                                        │  │
│  │  - CORS Configuration                                    │  │
│  └──────────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
                            │
                            │ Mongoose ODM
                            │
                            ▼
┌──────────────────────────────────────────────────────────────┐
│                         DATA TIER                            │
│  ┌──────────────────────────────────────────────────────────┐  │
│  │  MongoDB Database (Port 27017)                           │  │
│  │  - users Collection                                      │  │
│  │  - flights Collection                                    │  │
│  │  - bookings Collection                                   │  │
│  │  - Indexes and Relationships                             │  │
│  └──────────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
```

7. API Endpoints Summary

Method	Endpoint	Purpose	Auth Required
POST	/register	User registration	No
POST	/login	User login	No
GET	/fetch-flights	Get all flights	No
POST	/add-Flight	Add new flight	Yes (Operator)
POST	/book-ticket	Book flight ticket	Yes (Customer)
POST	/approve-operator	Approve operator	Yes (Admin)

8. Security Implementation

Security Feature	Implementation
Password Encryption	bcrypt with 10 salt rounds
Authentication	Session-based with localStorage
Authorization	Role-based access control (RBAC)
Input Validation	Client and server-side validation
CORS Protection	Configured allowed origins
Data Validation	Mongoose schema validation

9. Deployment Configuration

Component	Port	URL
React Client	3000	http://localhost:3000
Express Server	6001	http://localhost:6001
MongoDB	27017	mongodb://localhost:27017

10. Technology Justification

Why MERN Stack?

- **JavaScript Everywhere:** Single language for full-stack development
- **JSON Data Flow:** Seamless data transfer between layers
- **High Performance:** Non-blocking I/O and efficient rendering
- **Scalability:** Easy to scale horizontally
- **Community Support:** Large ecosystem and resources
- **Modern Development:** Latest web development practices
- **Cost-Effective:** Open-source technologies

© 2026 SB FlightConnect - Technology Stack Documentation